DAA Assignment 1

NAME: G Venkata Ganesh

Reg.no.: 192425381

Course Code: CSA0603

Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

At least **one iterative approach**
At least **one recursive approach**
A clear explanation of the basic idea behind each approach
Pseudocode or an appropriate flowchart for each approach
A brief justification for selecting each approach for investigation
Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation **2 Marks**

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

What the basic operation is
Where it occurs in the algorithm
Why it is appropriate to use this operation for efficiency analysis
How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

The input size (n)
The number of times the basic operation executes
How the operation count changes with (n)
The mathematical expression representing the running time
Any preprocessing cost involved
The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

What input condition produces the **best case**
What input condition produces the **average case**
What input condition produces the **worst case**
How the number of operations changes in each case
Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis **3 Marks**

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
How each algorithm behaves as (n) becomes very large
How the growth rates of the proposed approaches differ
What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation **3 Marks**

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

The input size of the original problem
The size of the recursive subproblem
The number of recursive calls
The work performed outside the recursive call
The base case
How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation **4 Marks**

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

Substitution method
Iteration method
Master Theorem
Another appropriate analytical technique

Discuss:

Why the selected method is applicable
The steps involved in solving the recurrence
The resulting complexity
What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms **4 Marks**

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

All algorithms produce the correct result
The same or equivalent datasets are used for comparison
Testing conditions are kept reasonably consistent
Appropriate test cases are included
The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance **3 Marks**

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

Execution time
Number of comparisons
Number of key operations
Input size
Other relevant performance measures, where justified

Present the results using appropriate:

Tables
Graphs
Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour **4 Marks**

Compare your theoretical analysis with your experimental results.

Discuss:

Whether the observed growth agrees with the theoretical complexity
Which algorithm scales better as (n) increases
Any differences between theoretical prediction and actual execution
Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

Which approach you recommend
Why it is more appropriate than the alternatives
Its theoretical efficiency
Its experimentally observed performance
Any preprocessing cost involved
Relevant trade-offs or limitations
Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design **8 Marks**

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

What has changed from the original problem
Which original assumptions are affected
Why the frequency of searches is now important
Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

Its expected behaviour under 50,000 searches per hour
Whether its preprocessing cost is acceptable under the new conditions
Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

Describe the revised algorithmic strategy
Explain what has changed
Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

Mathematical complexity analysis
Experimental results, where appropriate
Search frequency
Preprocessing cost
Dataset size
Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**

Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

SOLUTIONS

Topic: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Algorithms covered: Linear Search, Recursive Binary Search, and Hash Table Search.

Task 1: Design the Algorithm

1. Linear Search – Iterative

Idea: Check each registration number one by one until the required number is found.

Pseudocode:

```
LINEAR_SEARCH(A, key)
    for i = 0 to n-1
        if A[i] == key
            return i
    return -1
```

Strategy: Iterative searching. Preprocessing: Not required. It works with unsorted data but may inspect every record.

2. Binary Search – Recursive

Idea: Divide a sorted list into halves and search only the relevant half.

Pseudocode:

```
BINARY_SEARCH(A, key, low, high)
    if low > high: return -1
    mid = (low + high) // 2
    if A[mid] == key: return mid
    if key < A[mid]:
        return BINARY_SEARCH(A, key, low, mid-1)
    return BINARY_SEARCH(A, key, mid+1, high)
```

Strategy: Divide and conquer. Preprocessing: Sort records if needed. Search time is logarithmic when records are sorted.

3. Hash Table Search

Idea: Store registration numbers as keys in a hash table for fast lookup.

```
HASH_SEARCH(H, key)
    if key exists in H: return H[key]
    return NOT_FOUND
```

Strategy: Hashing. Preprocessing: Build the hash table. Lookup is expected O(1) with effective hashing.

Task 2: Identify the Basic Operation

• Linear Search: Compare A[i] with the target key.

• Binary Search: Compare A[mid] with the target key.

• Hash Search: Compute/use the key's hash and check the corresponding entry.

These operations dominate search time and are useful for comparing the algorithms.

Task 3: Derive Time Complexity Mathematically

A. Linear Search
Worst case: all n records are checked.

$T(n) = c_1 n + c_2$, so $T(n) = \Theta(n)$.

If the key is equally likely to be at any position, average comparisons = $(1 + 2 + \ldots + n)/n = (n + 1)/2$.
Preprocessing cost: 0.

B. Binary Search
$T(n) = T(n/2) + c$. After k divisions, $n/2^k = 1$, so $k = \log_2 n$. Therefore, search time is $\Theta(\log n)$.

If records are unsorted, sorting takes $\Theta(n \log n)$, so the total cost for one search is $\Theta(n \log n)$ including sorting.

C. Hash Table Search
Building the hash table takes expected $\Theta(n)$ time. Each lookup takes expected $\Theta(1)$. For q searches, total expected cost is $\Theta(n + q)$. Worst-case lookup can be $\Theta(n)$ when many keys collide.

Task 4: Best, Average and Worst Cases

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Search	$\Theta(1)$	Expected $\Theta(1)$	$\Theta(n)$

Linear search is best when the first item matches. Binary search is best when the middle item matches. Hash lookup is typically constant-time but can degrade due to collisions.

Task 5: Asymptotic Analysis

• Linear Search: $O(n)$, $\Omega(1)$, worst-case $\Theta(n)$.

• Binary Search: $O(\log n)$, $\Omega(1)$, worst-case $\Theta(\log n)$.

• Hash Search: Expected $\Theta(1)$ lookup; worst-case $O(n)$.

For one million records, linear search may inspect up to 1,000,000 entries. Binary search needs at most about 20 comparisons. Hash search is usually fast after construction.

Task 6: Formulate the Recurrence Relation

For recursive binary search, one recursive call handles about half the input, and the remaining work is constant.

$T(n) = T(n/2) + c$, for $n > 1$

$T(1) = c_0$

The base case occurs when the key is found or low > high.

Task 7: Solve the Recurrence Relation

Starting with $T(n) = T(n/2) + c$:

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

After k steps: $T(n) = T(n/2^k) + kc$. When $n/2^k = 1$, $k = \log_2 n$.

Therefore, $T(n) = T(1) + c \log_2 n = \Theta(\log n)$.

Task 8: Implement and Test the Algorithms

Run this Python program in IDLE. It tests the algorithms on 1,000, 10,000, 100,000, and 1,000,000 records.

```
import time

def linear_search(a, key):
    for i, x in enumerate(a):
        if x == key:
            return i
    return -1

def binary_search(a, key):
    low, high = 0, len(a) - 1
    while low <= high:
        mid = (low + high) // 2
        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def hash_search(h, key):
    return h.get(key, -1)

sizes = [1000, 10000, 100000, 1000000]
print("Size | Linear(s) | Binary(s) | Hash lookup(s)")

for n in sizes:
    records = list(range(1, n + 1))
    key = n

    start = time.perf_counter()
```

```
r1 = linear_search(records, key)
t1 = time.perf_counter() - start

start = time.perf_counter()
r2 = binary_search(records, key)
t2 = time.perf_counter() - start

start = time.perf_counter()
table = {x: x for x in records}
build_time = time.perf_counter() - start

start = time.perf_counter()
r3 = hash_search(table, key)
t3 = time.perf_counter() - start

assert r1 == n - 1 and r2 == n - 1 and r3 == key
print(n, "|", round(t1, 6), "|", round(t2, 6), "|", round(t3, 6))
print("Hash construction time:", round(build_time, 6), "seconds")
```

Task 9: Record Experimental Performance

Fill the table with the actual values printed by your computer. Do not invent timing values.

Dataset size	Linear time (s)	Binary time (s)	Hash lookup time (s)
1,000	5.46×10^{-5}	6×10^{-6}	2×10^{-6}
10,000	0.000493	5.7×10^{-6}	2.2×10^{-6}
100,000	0.00367	7.6×10^{-6}	1.5×10^{-5}
1,000,000	0.037759	1.44×10^{-5}	6.3×10^{-6}

Hash construction time is printed separately because it is preprocessing, not lookup time.

Task 10: Compare Theory with Experiment

• Linear search becomes slower as the dataset grows because it checks records sequentially.

• Binary search scales better because it repeatedly halves the search interval.

• Hash lookup is generally fast after the hash table has been constructed.

• Timing differences are normal due to hardware, Python, memory and system load. The results should broadly match theoretical trends.

Task 11: Recommend and Defend the Final Design

Recommend hash table search for frequent student record lookups.

1. Expected lookup time is O(1).

2. It is suitable for repeated searches across one million records.

3. New records are added only once daily, so the index can be updated incrementally.

4. It uses additional memory and must handle collisions and duplicate keys correctly.

5. Validate the recommendation using measured times, hash construction cost and memory usage. Binary search is a good alternative for sorted records.

Task 12: Re-evaluate and Adapt the Design

A. Impact Analysis

The system has one million records and 50,000 searches per hour. New records are added once per day, so search efficiency matters more than frequent index rebuilding.

B. Evaluate the Original Design

Hash table search remains appropriate because expected lookup time is O(1). Building the table takes expected O(n) time, which can be spread across many searches.

C. Adaptation or Redesign

Retain and optimize the hash table: build the index when records are loaded, insert new records when added, handle collisions and duplicate keys, and keep the index synchronized with the database.

D. Evidence-Based Defence

50,000 searches per hour is about 13.89 searches per second. A hash table can handle this workload efficiently under typical conditions. Confirm the choice with measured timings, memory requirements, collision handling and index maintenance costs.

Conclusion

Linear search is simple but may be slow for large datasets. Binary search is efficient for sorted data. Hash table search is recommended for repeated student record lookups because it provides expected constant-time access, provided the index is maintained correctly.